

Increasing throughput in dense 802.11 networks by automatic rate adaptation improvement

Kleber V. Cardoso · José F. de Rezende

Published online: 24 September 2011
© Springer Science+Business Media, LLC 2011

Abstract Rate control algorithms for commercial 802.11 devices strongly rely on packet losses for their adaptation. As a result, they give poor performance in dense networks because they are not able to distinguish packet losses related to channel error from packet losses due to collision. In this paper, we evaluate automatic rate adaptation algorithms in IEEE 802.11 dense networks. A certain number of works in the literature address this problem, but they demand modifications of the IEEE standard, or depend on some special feature not available in off-the-shelf devices. In this context, we propose a new automatic rate control algorithm which is simple, easy to implement, standards-compliant, and well-suited for crowded 802.11 networks. Our approach consists of measuring the contention level, inferring the collision probability, and choosing transmission rates which maximize throughput. Results from simulation and real experiments show throughput improvement of up to 100% from our mechanism.

Keywords 802.11 · Rate control · Link adaptation · Dense networks · Scalability · Congested networks

1 Introduction

IEEE 802.11 has become the *de facto* standard for the increasing market of wireless local area networks (WLANs). In addition to desktops and laptops, many other types of equipment are ready to communicate using 802.11 interfaces; these include printers, cameras, storage devices, and mobile phones. Hence, it is becoming common to find scenarios with many devices contending for medium access, also known as dense networks. In several situations, there are many networks or communicating pairs competing in the same channel. In this context, it is very important to choose algorithms (or mechanisms) that use the available bandwidth in an optimal manner. Automatic rate adaptation is one of these mechanisms.

The IEEE standard does not specify a rate adaptation algorithm, so manufacturers can make different implementations given that they do not disrupt other parts of the standard. Actually, a rate control mechanism must deal with limited information in order to stay compliant with the standard. For example, a mechanism cannot rely on the feedback of the signal-to-interference-plus-noise ratio (SINR) from the receiver. Despite the restrictions, there are many approaches to developing rate adaptation algorithms.

Choosing the best transmission rate is not a simple task. It is necessary to take into account channel conditions, which can vary significantly with time, without having enough information about the channel. Theoretically, the signal quality sensed by the receiver on every frame is what matters in choosing the transmission rate. Nevertheless, concerning 802.11b(g) networks, the standard does not provide any means for the sender to collect this information.

K. V. Cardoso (✉)
Instituto de Informática (INF), Universidade Federal de Goiás
(UFG), Goiânia, GO, Brazil
e-mail: kleber@inf.ufg.br

J. F. de Rezende
Grupo de Teleinformática e Automação (GTA), COPPE,
Universidade Federal do Rio de Janeiro (UFRJ),
Rio de Janeiro, RJ, Brazil
e-mail: rezende@gta.ufrj.br

There are several algorithms for rate adaptation in 802.11 networks [12, 18, 21, 25, 28, 38, 42, 43, 46, 52], but only two of them have been widely cited because they are used in commercial devices [9, 27].¹ Even though the number of algorithms is large, there is still considerable room for improvement since there is no optimal solution to all scenarios. In addition, the great majority of proposals do not consider the limitations enforced by the hardware, leading to poor performance when they are implemented in real devices [44]. Some proposals do not adhere to the IEEE standard, making their adoption less likely.

In this paper, we deal with the problem of distinguishing between frame losses caused by link errors and frame losses caused by collisions, which are quite common in dense networks. Losses due to collision must not affect the rate selection as opposed to losses by signal degradation. In order to treat this issue, we propose an algorithm that measures the channel occupation level and infers the probability of a loss occurring due to a collision. In contrast with other approaches, our mechanism does not violate any IEEE standard rule, and takes into consideration hardware constraints of commercial equipments.

The main contribution of this paper is twofold. First, it provides an easily implementable 802.11 rate control algorithm that obtains an outstanding performance in both sparse and dense environments. Our solution is fully IEEE 802.11 standards-compliant and it can be implemented in any off-the-shelf 802.11 devices. Moreover, the paper presents a large number of performance results obtained on real experiments involving known algorithms such as ARF and Sample Rate.

This paper is organized as follows. In Sect. 2, automatic rate adaptation is reviewed in more detail, some important mechanisms are described and some additional comments about dense networks are provided. Section 3 presents our new algorithm and describes how it handles the frame losses caused by the intense contention encountered in dense networks. Section 4 describes the performance results of our mechanism, and compares our results with results of other algorithms. In Sect. 5, the paper is concluded and we discuss some potential extensions to this work.

2 Automatic rate control algorithms

The IEEE 802.11 standard states that compliant devices must support multiple transmission rates at the physical layer (PHY). Each PHY rate represents a different

combination of modulation techniques and coding rates. For instance, 802.11b must provide four transmission rates (1, 2, 5.5, and 11 Mbps), while 802.11g must deal with eight additional rates (6, 9, 12, 18, 24, 36, 48, and 54 Mbps). In theory, as the transmission rate decreases, noise immunity increases. This means that as the PHY rate decreases, the signal becomes more robust against noise, so the receiver can decode frames with a lower SINR. In other words, two different PHY rates show distinct frame loss probabilities as functions of the same SINR. Basically, rate adaptation consists of reducing the transmission rate when channel quality becomes worse, and augmenting it when the quality becomes better. In this context, channel quality is related to the SINR observed by the receiver while receiving a frame. This channel quality may vary significantly over short time intervals due to multipath, shadowing, mobility, interfering signals, etc. It is interesting to verify that SINR can vary widely even in static networks. We have confirmed this behaviour by a large number of experiments in our laboratory and in two home environments as described in [10]. While the nodes of the networks stayed fixed during the tests, many objects move around them, making the SNR to change widely. Similar scenarios can be commonly found in the real world, e.g. in airports, restaurants, meeting rooms, coffee shops, shopping malls, etc. Rate control algorithms take different approaches to cope with the link quality instability, and usually try to maximize the available throughput.

IEEE 802.11b and .11g do not specify a means by which the receiver informs the sender about the channel quality (or SINR). Because of this constraint, many algorithms (e.g. [9, 27, 33, 52]) follow an open-loop rate adaptation approach, which is based only on losses detected by the sender (by not receiving the ACK from the receiver). However, some authors propose a closed-loop approach (e.g. [21, 24]) by circumventing the lack of information from the receiver. The latter class of algorithms requires changes in the MAC header format and/or addition of new frames to carry SINR-related information. This is an IEEE 802.11 standard violation, and so it is not adopted by manufacturers. In addition, it requires that both sides of a communication use the same mechanism to provide a performance benefit, thus limiting its use in heterogeneous environments.

A different approach [26, 41, 53, 54] assumes that 802.11 links are symmetric and make SINR-like measurements at the sender. Hence, it is an open-loop solution, but it infers additional information about the channel. SINR measurements may use only ACK frames, but they can also use data, control, and management frames. Nevertheless, this approach has problems which are hard to solve and can cause significant performance degradation. First, many 802.11 links are asymmetric [32, 40]. Second, the sender

¹ Unpublished algorithms are trade secrets.

needs to know the output power of the receiver, which is not available in a standards-compliant way. Finally, the SINR can vary significantly over short time intervals [52] and hardware can present different correlations between SINR values and loss probabilities for every PHY rate [3]. All these issues make the calibration of a mechanism based on channel quality very complex.

Table 1 summarizes the previous assortment of rate adaptation algorithms. In this table are listed some additional works that will be commented in the following.

The first published work on automatic rate adaptation in 802.11 networks concerned Auto Rate Fallback (ARF), which was originally proposed for use in WaveLAN-II technology [27]. Some papers [13, 28, 29, 39, 46] claim that this algorithm, or some variant, would be employed in several commercial devices. Basically, ARF increases the transmission rate when it receives ten consecutive ACKs, and decreases the PHY rate when two sequential losses are seen or a loss occurs just after a rate increment. This simplistic approach creates two opposing drawbacks. On one hand, the algorithm is very conservative because it reduces the rate when it finds only two consecutive losses. Wong et al. [52] shows that there is a high probability that two losses occur in sequence at any PHY rate. On the other hand, ARF is very aggressive since it frequently keeps trying to augment the rate (on every ten consecutive ACKs), even if there is a long history of unsuccessful attempts. For every retransmission retry of the same frame, there can be a long delay due to the increment in the backoff window, which will be explained in more detail in Sect. 3.

Another key algorithm is SampleRate [9]. It is the default mechanism for a GNU/Linux open driver [36]. SampleRate is a mechanism that monitors the mean transmission time at each PHY rate, privileging the rate with the lowest delay; i.e., the one with the highest throughput. Traditionally, the transmission rate is increased when the link quality improves, and is decreased when it degrades. SampleRate extends this concept by assuming that, eventually, a higher PHY rate can be better than the present one, even when the latter starts to sense channel degradation. This idea is supported by the fact that 802.11 allows for combinations of modulations and coding rates that do not show the same robustness relationship to certain wireless channel problems.

For example, depending on the type of the interference, it may be more efficient to transmit at 11 Mbps instead of 5.5 Mbps. The focus is kept on the throughput, and therefore losses are tolerable. For instance, transmitting at 5.5 Mbps with a loss rate of 10% is better than transmitting at 2 Mbps without losses.

Many other rate adaptation algorithms have been proposed in the literature, which exhibit good performance in several scenarios. However, most of these algorithms present poor performance in dense networks. Some works have already tackled the problem of rate adaptation in networks with many stations contending for medium access [2, 17, 19, 28, 50]. The problem is the incidence of a large number of collisions when there are several stations trying to transmit, regardless of whether they are communicating with their APs or exchanging frames in *ad hoc* mode. For open-loop and loss-based approaches—the types of algorithms widely adopted by the device manufacturers—the issue is how to identify the nature of losses; i.e., collision or channel error. If the link quality is good enough, frame losses must not lead to a decrease in the transmission rates; rather, they need to be maintained or even increased. Under high contention and good link quality, the best choice is to transmit at the highest rate that the link quality allows, regardless of collision-related losses. This approach maximizes the overall network (or aggregated) throughput.

Kim et al. [28] presents an algorithm called Collision-Aware Rate Adaptation (CARA), which is based on RTS/CTS messages. The algorithm assumes that data frames can be affected either by channel errors or by collisions, but RTS/CTS frames are frequently lost by collisions. However, a CARA-like algorithm was implemented in the MadWifi driver by Ramachandran et al. [44], and its aggregated throughput was much lower than those found in theoretical and simulation results. According to the authors, this difference between simulation and experimental results probably comes from implementation issues with RTS/CTS that result in data frame losses even when the channel is reserved and is not limited by SINR. Some other works [5, 7, 8, 11, 13, 20, 35, 39, 48, 51] are prone to a similar problem, since they were implemented only in a simulator and made strong assumptions that are hard to confirm using real hardware.

In [1, 2], Wireless cOngestion Optimized Fallback (WOOF) is proposed, which uses channel busy time as a measure of contention in the wireless medium. However, this metric fails in many scenarios, so the authors introduced a confidence factor to measure the degree of correlation between the channel busy time and collision-related packet losses. The drawbacks of this approach are the application of a metric that is completely unreliable, and the need to continuously evaluate its behavior. There is no

Table 1 Types of rate adaptation algorithms

Open-loop	Closed-loop	Open-loop + SINR
[1, 2, 9, 13, 17, 27, 33, 39, 48, 51, 52], this work	[20, 21, 24, 28]	[5, 8, 11, 19, 26, 41, 50, 53, 54]

problem when the metric makes a good guess, but in the opposite situation, time is wasted while taking an incorrect action. This shortcoming becomes worse in environments with varying levels of medium contention. The authors implemented and evaluated their mechanism only in a small infrastructured network with no more than eight clients. The same limited number was used with Snoopy Rate Adaptation (SRA) in [17], which has the additional drawback of assuming that providing the total number of transmissions and the number of contending nodes which are active during the last 200 ms interval is costless and widely supported by 802.11 devices.

In summary, the present solutions have one or more of the following weaknesses: they are only evaluated in small-size networks, they are unsuccessfully implemented, and they are not implemented at all because of hardware and/or standards constraints. Our mechanism, called Yet Another Rate Adaptation algorithm (YARAA), overcomes all of these limitations, and is described and evaluated in the next sections.

3 Yet another rate adaptation algorithm

YARAA is a throughput-based algorithm that inherits characteristics from SampleRate and adds the ability to deal with the problem of frequent collisions, which are common in dense networks. From SampleRate, YARAA keeps the transmission time estimation method and the ability to switch between non-adjacent rates. Our mechanism also monitors the effective transmission time and the highest successful transmission rate in the recent past. As we will discuss in this section, the effective transmission time helps to identify the level of contention in medium access and to infer the collision probability.

Figure 1 shows the time diagram of one station transmitting in the basic access mode of 802.11, as known as the distributed coordination function (DCF). Only the events related to rate adaptation are described below. As seen in the figure, a station A, before transmitting, monitors the transmission medium until it is idle; then, the station waits for DCF inter-frame space (DIFS) plus a backoff. The

backoff value is randomly chosen within the contention window (CW) interval. The backoff counter is decremented while the medium is idle, it is stopped when other stations transmit, and it is resumed when the medium becomes idle again. If station A does not detect any transmission while decreasing backoff counter, there is a continuous idle medium time equal to the chosen backoff value. Every time the decrement of backoff value is stopped, due to a carrier sense, the idle medium time is fragmented. In general, between two idle medium time fragments there is one or more frame transmissions which are not taken into account by IEEE 802.11 standard. As a consequence, the actual deferred transmission time can be equal to the backoff time or very different, depending on the number of stops the backoff counter suffers.

As the number of stations trying to transmit increases, the number of times the backoff counter is stopped increases. In other words, as the contention level rises also increases the difference between the chosen backoff value and the deferred transmission time. Although not depicted in the figure, the frame from station A could not be received due to a collision, requiring a frame retransmission. For every frame retransmission a new backoff value is chosen, basing on an exponentially augmented CW. However, this does not avoid the influence of the contention level on the difference between chosen backoff value and actual deferred transmission time. The collision probability increases as the number of contending stations becomes larger [6, 31, 49], and this affects the loss-based rate adaptation algorithms [44].

In order to compute the throughput for each PHY rate, YARAA uses the transmission time (tx_time), illustrated in Fig. 1, which is described by the following estimate [9]:

$$tx_time(b, r, n) = DIFS + \sum_{i=0}^r backoff(i) + (r+1) * (SIFS + ACK + header + (n * 8/b)) \quad (1)$$

where b (bit-rate) is the transmission rate of the physical layer, r (retries) is the number of eventual retransmissions

Fig. 1 Basic access mode of 802.11 networks (DCF)

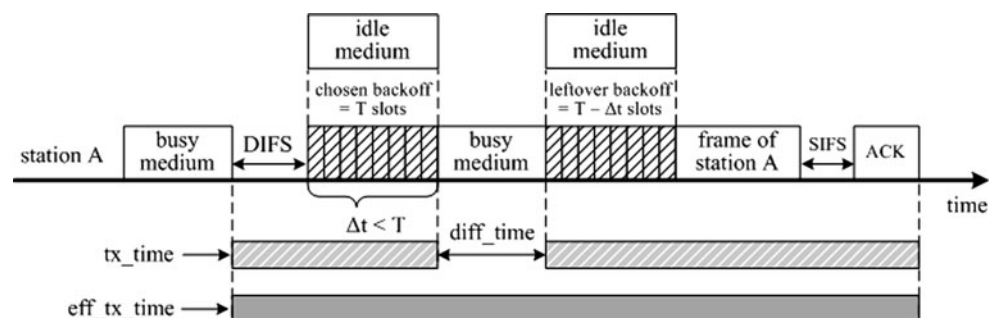


Table 2 Some parameters of IEEE 802.11a/b/g

Standard	Slot	DIFS	SIFS	ACK	Header
802.11a	9 μ s	34 μ s	16 μ s	Header + 14*8/b'	20 μ s
802.11b	20 μ s	50 μ s	10 μ s	Header + 14*8/b'	192 μ s (long preamble) or 96 μ s (short preamble)
802.11g (only)	9 μ s	28 μ s	10 μ s	Header + 14*8/b'	20 μ s

until a successful reception, and n (n -byte) is the frame size in bytes. The backoff assumes the value of the contention window average for the specific number of retries according to the IEEE standard; i.e., the minimum window size is 15 (.11a) or 31 (.11b/g) slots, and the maximum is 1,023 slots. The remaining parameters are described in Table 2. The ACK PHY rate is given by b' , and it is a value less than or equal to the PHY rate of the corresponding data frame [22].

The estimated transmission time (**tx_time**) is able to measure the impact of retransmissions over throughput. By computing **tx_time**, the algorithm can measure the burden of channel errors [9], but it is not able to distinguish this type of loss from collisions. To overcome this limitation, YARAA additionally keeps track of the effective transmission time (**eff_tx_time**), which denotes the time from the frame arriving at the interface queue head until the MAC layer is notified of a successful transmission. This value can be very different from the estimated transmission time (**tx_time**), since the latter does not take into account the time during which the backoff counter remains frozen due to the busy medium. Finally, the difference between these two time intervals is computed (**diff_time**).

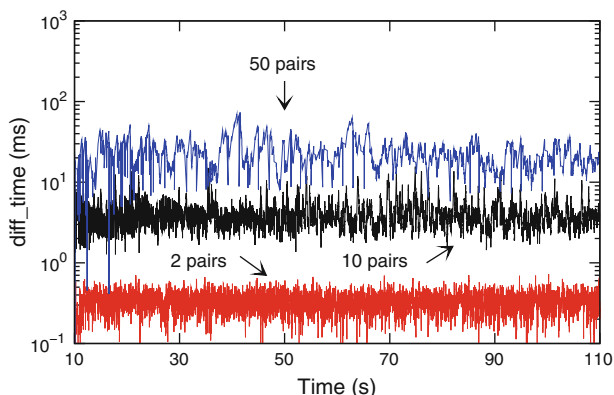
During the time in which a few stations are trying to transmit, **tx_time** is a good estimate of the effective transmission time. Nevertheless, as medium access contention increases, there is an increase in the difference (**diff_time**) between the effective and estimated times. This behavior is illustrated in Fig. 2. YARAA uses the changes in **diff_time** values to identify the level of contention and

to infer the probability of a loss being caused by collision. The number of times the backoff counter is stopped could also be used as a contention level indicator. However, to the best of our knowledge, this information is not available for the driver in commercial devices. By loss differentiation, it is possible to minimize incorrect rate adaptations and thereby improve network throughput.

Figure 2 shows **diff_time** values as function of the simulation time for increasing levels of contention for medium access. These levels are represented by different numbers of station pairs exchanging frames. For the sake of readability, only values for 2, 10, and 50 pairs are presented here. In this evaluation, all traffic sources are saturated; i.e., they always have data to send. The number of transmitting stations producing the levels of contention shown in Fig. 2 can be different when sources are not saturated. However, this does not affect YARAA's performance since the algorithm does not rely on the exact number of transmitting stations. In Sect. 4, **diff_time** is revisited in the context of real network environments, and it is confirmed as a good indicator of the contention level.

Listing 1 summarizes how YARAA works. The **tx_time** and **eff_tx_time** variables are the exponentially-weighted moving averages (EWMAs) of the instantaneous values. The EWMA value is used due to the high variance in the samples, which could lead to algorithm instability and performance degradation. The **diff_time_lowThresh** and **diff_time_highThresh** thresholds are set according to the values found in the **diff_time** measurements (see Fig. 2).

If the contention level (**diff_time**) surpasses the first threshold (**diff_time_lowThresh**), YARAA starts using a probability to decide whether the next frame should be sent at the highest successful transmission rate. As the contention level for medium access rises above this threshold, the probability of sending the next frame at the highest successful transmission rate is increased. If it reaches the second threshold (**diff_time_highThresh**), the probability is set to 1. When the mechanism chooses not to try the highest successful PHY rate, then the rate is determined in a way similar to SampleRate; i.e., by preferring the rate with the lowest estimated transmission time. The idea of two thresholds and a varying probability between them comes from the Random Early Detection (RED) mechanism [15], which is proposed for gateways in packet-switched networks to avoid congestion.

**Fig. 2** Difference between effective and estimated transmission time in an 802.11g network

Listing 1 YARAA's algorithm

```

1   diff_time = eff_tx_time - tx_time;
2   prob_high_rateTry = (diff_time - diff_time_lowThresh) /
3                       (diff_time_highThresh - diff_time_lowThresh);
4   if (diff_time <= diff_time_lowThresh) {
5       highRateTry = false;
6   } else if (diff_time >= diff_time_highThresh) {
7       highRateTry = true;
8   } else {
9       randomValue = Random::uniform(0,1);
10      if (randomValue <= prob_high_rateTry) {
11          highRateTry = true;
12      } else { highRateTry = false; }
13  }
14  if (highRateTry) { selectHighestRate(); }

```

The highest successful transmission rate is not necessarily the highest PHY rate of the technology in use (e.g., 54 Mbps for 802.11g). Due to link quality, it can be necessary to choose a lower PHY rate (for example, 48 Mbps). This is very important to obtain improved performance with YARAA in dense networks under different channel conditions. Also, it is important to note how often this special highest PHY rate is updated. The interval can vary from tens of milliseconds to 60 s, depending on the amount of traffic being generated by the station. As the sending rate of a station grows, the information about transmission rates becomes more accurate.

The **selectHighestRate()** acts as the following every time it is called. The PHY transmission rates are checked in a decreasing order and the first available one is returned. A PHY rate becomes unavailable when the transmission of frames are unsuccessful three times in sequence, since all three tries occur in a time window of 60 s. An unavailable PHY rate becomes available again after 10 s.

Since practical implementation issues were taken into account in the YARAA design, YARAA is feasible for embedding in off-the-shelf 802.11 devices. In summary, YARAA only uses information easily obtained from the devices, such as transmission time. It does not depend on RTS/CTS messages, and it does not require any change in existing control frames or additional ones; i.e., the algorithm stays fully compliant with the IEEE standard. YARAA presents better performance than other mechanisms which are implemented in commercial devices, as we will demonstrate in the next section.

4 Performance evaluation

This section presents a performance evaluation of YARAA and some other well-known adaptation rate algorithms. Results obtained from a large set of simulation tests are

shown. Then, performance results from MadWifi driver implementations in a large wireless testbed, including YARAA, are presented. At last, some additional experiments are presented to explore in more detail the behavior of YARAA and its features. Through these performance results, we demonstrate the improved performance of YARAA in both simulated and real environments.

4.1 Simulation

We used the network simulator ns-2 (release 2.31) to perform the simulation-based evaluation. To this aim, we have added new features to the simulator as follows:

- Addition of support for multiple PHY rates and a frame error model that takes into consideration the SINR of the received packet, through application of the library patch presented in [47].
- Addition of two channel fading models: Ricean [4] and shadowing [37].
- Implementation of the SampleRate algorithm according to the MadWifi driver code [36].
- Implementation of YARAA.

Since our evaluation concerns only physical and link layers, the DumbAgent of ns-2 is used as the routing agent. It establishes only one-hop communications; i.e., it disables forwarding, and it does not generate control messages.

In this section, the evaluation of four algorithms (IDEAL, ARF, SampleRate and YARAA) is presented. IDEAL is a perfect mechanism that cannot be implemented in a real device because it knows in advance (i.e., before sending the frame) the SINR it will have at the receiver. Based on this privileged information, IDEAL chooses, for every frame, the ideal rate to achieve a certain loss probability under any link condition. The IDEAL mechanism provides to the sender the SNR value of every frame in receiver, before the frame to be really transmitted. Based

on the SNR and on the desired frame error rate, IDEAL chooses the best modulation scheme, i.e. the highest PHY transmission rate that meets SNR and FER values. SNR can be computed a priori because in the simulator we have access to all necessary information, such as the nodes' positions and the propagation model. In order to get the highest rate in successful transmissions, IDEAL uses a value of FER near to zero.

The IDEAL mechanism assumes that before each transmission the SNR value that the corresponding frame will achieve at the receiver is known. Based on this information and on the desired frame error rate (FER), the sender can choose the best modulation to be used at each frame. The best modulation scheme is the one that provides the highest PHY transmission rate for the respective SNR while the required FER is met. The SNR can be computed a priori since, in the simulator, one can have access to all necessary information, such as the nodes' positions, the transmission power and the propagation model. In contrast, a value close to zero is chosen for FER in order to achieve the highest rate in successful transmissions.

The ARF implemented by the Padova library [47] has been modified to allow a per-link adaptation rate rather than all devices in the same link. In the original implementation, a unique PHY rate is used by the sender to communicate with multiple receivers. Therefore, the modified implementation individually computes the number of lost and successful frames for every communicating node, and maintains a PHY rate for each of them. All the other implemented algorithms have this same ability.

We show results for a packet size of 1,500 bytes, although different packet sizes were also evaluated and exhibited similar results. Each simulation run lasted for 110 s, and was executed 30 times for each parameterization. The mean values are presented with confidence interval bounds at a confidence level of 95%. All traffic sources have a random start time, which is uniformly distributed in

the interval comprised by the first 10 s of the simulation run. The initial 10 s are considered as transient and were discarded.

In the following, we present the simulation results divided into three parts. In the first two parts, we evaluate the performance of the algorithms on two diverse network topologies: one-hop *ad hoc* and infrastructured networks. The third evaluation concerns the impact of three different propagation models on the algorithms.

4.1.1 Communicating pairs

In this first set of tests, our goal is to evaluate the performance of the algorithms with different numbers of stations contending for access in a one-hop *ad hoc* network. In these tests, we used a frame error model based on SINR in which the signal power only decays as a function of the distance. As dense networks are commonly established in small areas, an area of 100 m² was used. The nodes (stations) were arranged in a grid topology with a minimum distance of 1 m, and each sender-receiver pair was chosen randomly. Due to the short distances among nodes, there are no simultaneous transmissions without collision; i.e., there is no channel reuse. TCP and UDP sources, both under saturated conditions, were evaluated. Both traffic types presented a similar trend in performance, only differing in their absolute values due to the TCP congestion control that reacts to losses and thus achieves a lower aggregated throughput than UDP. For the sake of brevity, only the results for TCP sources are presented.

Figure 3(a) and (b) illustrate how aggregated throughput varied as a function of the contention level for 802.11b and .11g, respectively. In an 802.11b network, SampleRate presented good performance when the number of contending pairs was less than 15, and suffered severe degradation from that point on. In an 802.11g network, SampleRate performance started degrading early, from

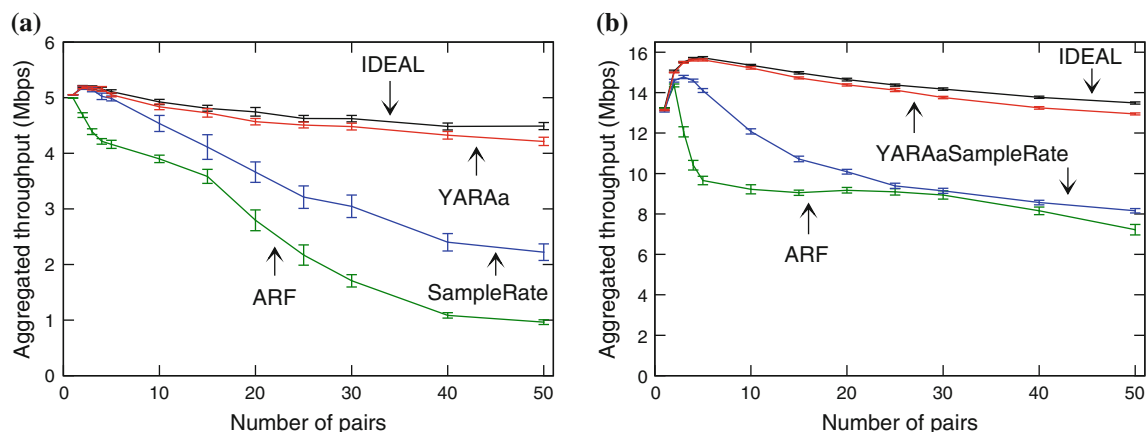


Fig. 3 Aggregated TCP throughput as function of number of communicating pairs. **a** 802.11b, **b** 802.11g

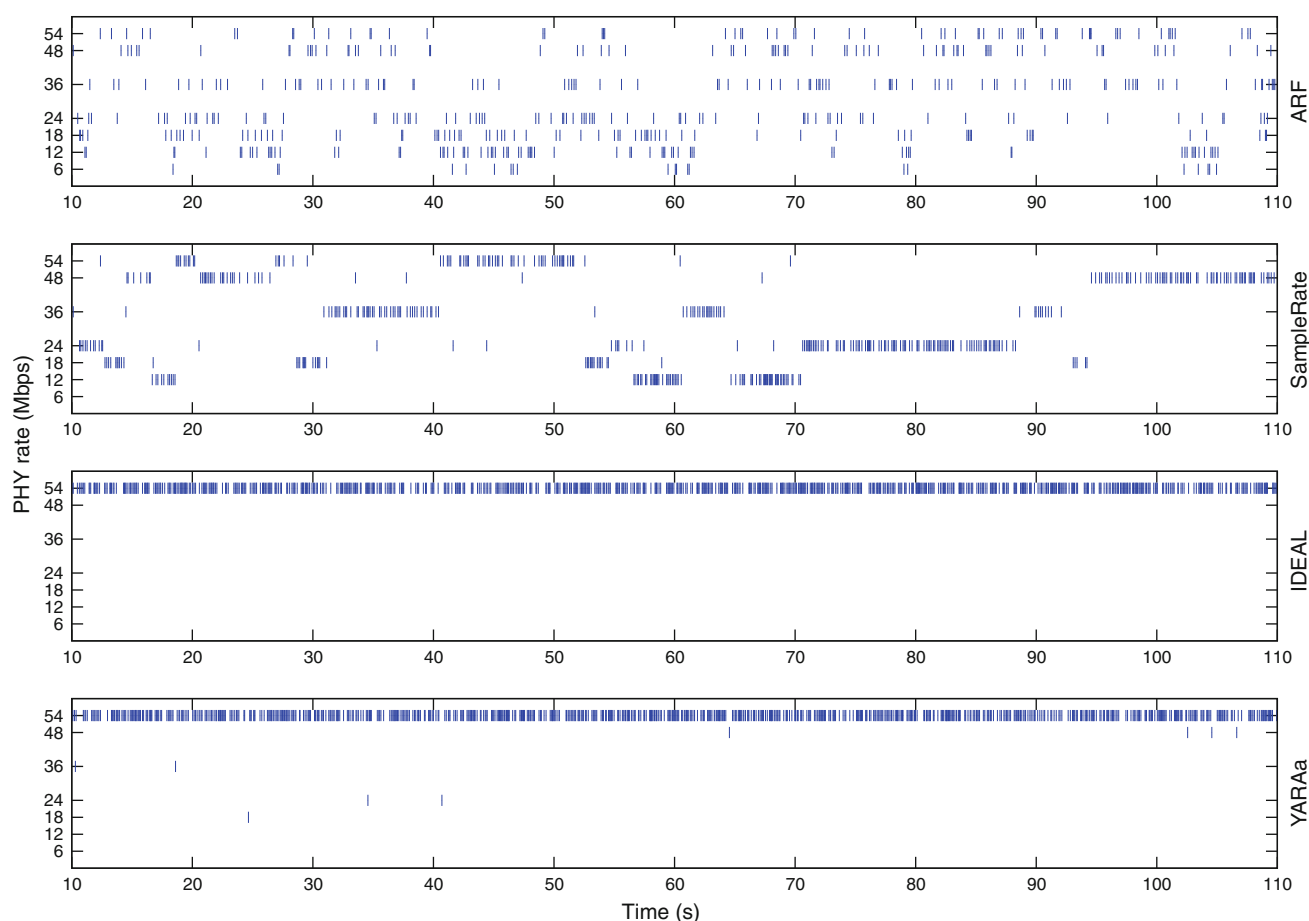


Fig. 4 A sample of PHY rate adaptation in different algorithms

only 4 pairs, and it became as inefficient as ARF when the number of pairs was greater than 20. As expected, ARF presented the worst general performance, while IDEAL achieved the maximum aggregated throughput. YARAA obtained throughput close to IDEAL, with a slight increase in the difference between them as the contention level increased. This difference comes from the burden of probing other PHY rates. However, it is important to keep this procedure, since it allows us to detect the best transmission rate for different channel conditions; i.e., different SINR values.

In Fig. 4, the rate adaptation is shown on a per-frame basis. Each graph shows the selected rate by a different algorithm used in the same station. This evaluation was made with 20 saturated stations. Theoretically, throughput-based algorithms (e.g. SampleRate) should deal properly with high contention levels but also with a high SINR because this type of algorithm gathers information from different PHY rates, and all rates suffer a similar amount of losses. However, when the contention level is high, the number of samples available is small, which leads to inaccurate statistics and incorrect adaptation decisions. The

figure also presents the frequent ARF switching among all rates due to the similar loss probability of each rate under the saturated state. As expected, the IDEAL mechanism always stayed tuned to the highest PHY rate. Finally, Fig. 4 shows why YARAA's results were so much closer to the IDEAL mechanism. Even in the presence of high loss rates, YARAA was able to identify the loss nature and adapt to the highest transmission rate.

4.1.2 Infrastructured network

This part of the evaluation is based on infrastructure mode, following a similar approach to other papers [2, 13, 17, 28, 44, 51]. The same frame error model based on SINR from the previous section was used in these tests. Again, the nodes were randomly distributed in a grid topology, always keeping the access point (AP) in the center of the area. 802.11b and .11g were evaluated, but only 802.11g results will be presented since the algorithms showed similar behavior in both technologies. TCP and UDP protocols were tested, but only UDP results will be shown. In infrastructure mode, there is a small differentiation among

the rate adaptation mechanisms when the traffic is fully TCP-dominated. At the end of this section, some additional comments regarding this matter are presented. Results are shown to a maximum number of 20 stations contending for medium access, which is enough to clearly distinguish the performance of the algorithms.

In Fig. 5(a), the traffic flows only from the stations to the AP (upstream), and all sources are saturated. We verified that SampleRate and ARF suffer severe performance degradation as the number of contending stations increased, while YARAA achieved an aggregated throughput very close to that obtained by the IDEAL mechanism. This evaluation methodology is widely employed for rate adaptation algorithms in dense networks because it offers a clear distinction among algorithms' performance with the smallest number of nodes. Nevertheless, previous works do not investigate the necessary ratio of upstream-to-downstream traffic to clearly differentiate the mechanisms. We present this result as follows.

Figure 5(b) illustrates how different percentages of upstream traffic influence the performance of the mechanisms. The total number of active stations is fixed at 20, and the number of upstream stations varies from 0 to 20. When the upstream traffic was lower than 30%, there was no difference in the performance of the algorithms. From 40% up, the aggregated throughput of ARF and SampleRate started deviating from IDEAL and YARAA. Downstream-dominant traffic implies low contention, since only the AP has data frames to send. As the number of uplink clients increases, the contention access, and hence the number of losses by collision, also rises.

In the scenario with single AP and TCP traffic only, the performance difference among the rate control algorithms becomes less significant. YARAA still achieved the best performance, and SampleRate and ARF give quite similar results. This behavior is related to two characteristics of TCP. First, traditional congestion control mechanisms

unfairly penalize the performance of TCP in wireless networks due to the higher number of channel error losses when compared with those occurring on wired networks. Second, the closed-loop approach of TCP creates a sort of serialization of clients in the AP, decreasing the contention level. Since all stations depend on only one AP to receive (or send) packets and the TCP congestion control detects this bottleneck, there is low contention for medium access. However, as the number of APs increases, this behavior changes, as shown in [14]. With more APs, the scenario becomes similar to the situation with communicating pairs described in Sect. 4.1.1, wherein only four pairs could provide a clear differentiation among the algorithms.

4.1.3 Propagation models

In this section, the rate adaptation algorithms are evaluated in the presence of losses from both collisions and channel errors. The channel errors were simulated using two additional propagation models in order to represent common problems of wireless communications; e.g., multipath and fast fading. This part of the evaluation employed a methodology similar to the one used in Sect. 4.1.1. We present results for the following setup: 802.11g technology, UDP traffic, and 20 saturated stations. The simulation experiments were repeated 60 times to reduce the confidence interval, since the chosen propagation models generate additional randomness in the simulations

The first set of experiments used the Ricean model [4], a fading model that describes channel conditions which vary in time. This model has two adjustable parameters: K and $maxVelocity$. K , or the Ricean K factor, is the ratio of the powers of the line of sight (LoS) component to the diffuse component, which indicates the relative strength of the LoS, and hence is a measure of link quality. The parameter $maxVelocity$ is the maximum speed of any object in the environment. The Ricean model is used to modulate the

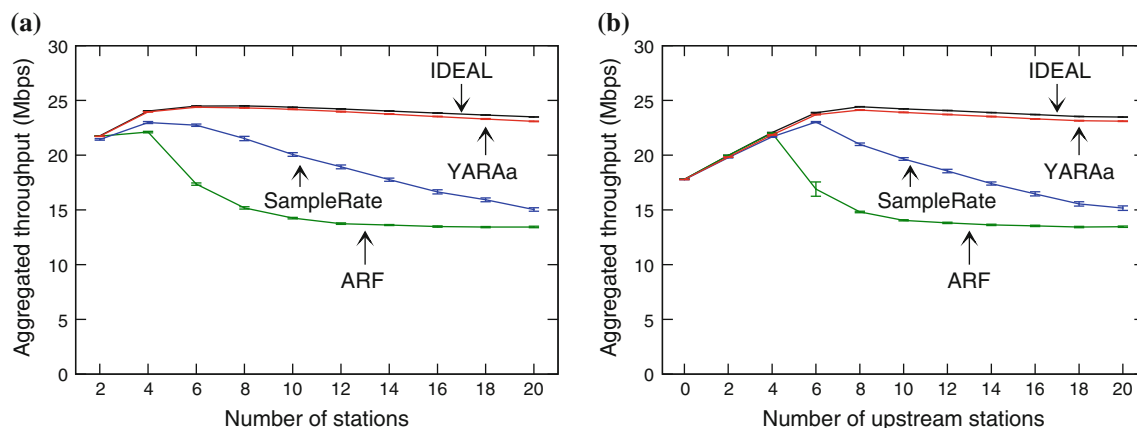


Fig. 5 Aggregated UDP throughput as function of number of stations.

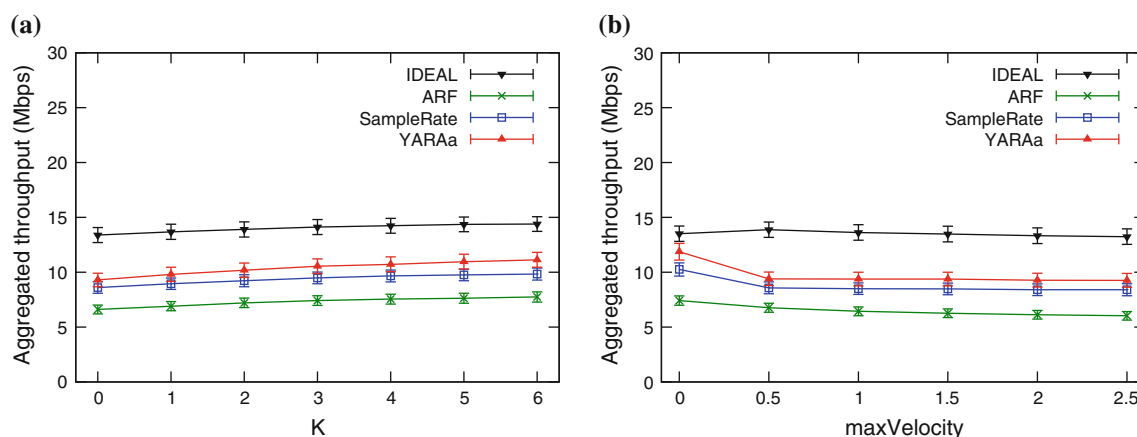


Fig. 6 Aggregated UDP throughput in a hostile environment. **a** Line of sight, **b** Objects mobility

output of a large scale fading model. In our simulation, this fading model is the two-ray ground, which represents a signal with two components—one straight and other reflected on the ground.

Depending on the values of the parameters, the Ricean model generates significant variations in signal power and creates additional channel errors. However, the simulated environment of our interest is represented by a small area with nodes close to each other; i.e., a typical dense network. In this kind of environment, the SINR is high in the receivers, and so the Ricean model does not increase frame losses until a level that representatively changes the previous behavior of the rate adaptation mechanisms. The results are almost indistinguishable from those presented in Sect. 4.1.1; therefore, we do not show a graphic since there is no new information

In the second set of tests, we applied the approach proposed in [37] which uses the shadowing model to represent large scale fading and keeps the Ricean model to describe time-varying fast fading. In the shadowing model, the received power at a certain distance is a random variable representing the multipath propagation effects. The shadowing model consists of two parts. The first is the path loss model, which predicts the mean received power at a specific distance and is configured by a parameter β , called the path loss exponent. The second part of the shadowing model describes the variation of the received power at a certain distance, and is set by σ_{dB} , called the shadowing deviation. Figure 6(a) shows results for $\beta = 4$ (obstructed indoor) and $\sigma_{dB} = 7$ (office, hard partition). Figure 6(b) presents results for the same $\beta = 4$, but $\sigma_{dB} = 9.6$ (office, soft partition). These two configurations illustrate hostile environments for wireless communications since they severely affect the SINR values.

Figure 6(a) shows the influence of K on the aggregated throughput, and Fig. 6(b) exhibits the influence of $maxVelocity$ on the same metric. Again, the Ricean model has a

small effect on the rate adaptation mechanisms. There is a slight increase in the aggregated throughput as K rises, which means an improvement in SINR (and hence a frame loss reduction). As objects start moving ($maxVelocity > 0$), the aggregated throughput of all mechanisms except IDEAL decreases due to high variations in channel quality. IDEAL is not affected because it can foresee the correct values of SINR, but the other mechanisms depend on a trial-and-error technique. Values of $maxVelocity$ higher than 0.5 have no effect. On the other hand, the shadowing model severely affects the performance of the mechanisms. The difference between the soft and hard partition is very small, but both make the SINR vary significantly. The aggregated throughput of all mechanisms is very much lower than the value obtained with the other models (two-ray ground and two-ray ground + Ricean). The performance of YARAA is still the second best, only lower than the IDEAL mechanism. In these extreme conditions, the difference between YARAA and SampleRate is smaller because the SINR varies broadly, and there are many losses from channel errors. Hence, YARAA spends most of the time searching and changing to the PHY rate that matches the SINR.

4.2 Field trial

We implemented YARAA as a rate adaptation module for the MadWifi driver [36] (release 0.9.4). Originally, MadWifi was a partially open driver for GNU/Linux and relied on a binary hardware abstraction layer (HAL), which hid most of the device-specific registers. Nowadays, the HAL code has been released by the chip manufacturer (Atheros).

In its original version, MadWifi does not measure the effective transmission time of a frame. Since this is fundamental information for YARAA, we implemented this measurement in the following way. In the procedure of sending a frame, the last task of the driver is to put the frame in the interface queue. At this moment, the current

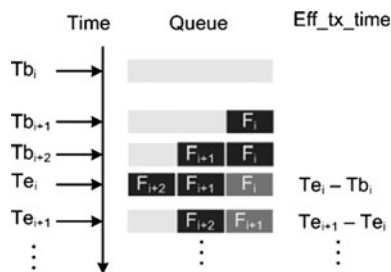


Fig. 7 Measurement of the effective transmission time

time is saved as the possible start transmission time (Tb_i) of frame i . This time is the actual time the transmission starts if no other frames are waiting for transmission completion when frame i arrives. When the hardware finishes transmitting a frame, it generates an interrupt that is treated by the driver, which sets the end transmission time (Te_i) of frame i to the current time. At this moment, if there is a next frame ($i + 1$) waiting in the queue, then the following inequality is true: $Tb_{i+1} < Te_i$. Hence, the frame $i + 1$ has a queuing time, due to the previous transmission, that should not be taken into account during its effective transmission time. Thus, in order to remove improper queuing time, the start transmission time of the next frame, if already in the queue, is replaced by the end transmission time of the previous transmitted frame ($Tb_{i+1} = Te_i$). Figure 7 illustrates the measurement of the effective transmission time of two frames, F_i and F_{i+1} .

In Atheros chips, there is a mechanism called multi-rate retry, which is configured by the rate adaptation algorithms. This mechanism allows every frame to be transmitted at different rates [30]. The multi-rate retry uses an array with four elements, each composed by the transmission rate and its associated retry counter (r_0/c_0 , r_1/c_1 , r_2/c_2 , r_3/c_3). The counter specifies the maximum number of retries to be used in the transmission of the frame at the associated rate. If this number is exceeded, the next element of the array, if it is available, is used in the subsequent retries. For example, the array (54M/3, 36M/2, 11M/1, 1M/1) indicates there will be, in this order, three tries at 54 Mbps, two tries at 36 Mbps, one try at 11 Mbps, and one try at 1 Mbps. Next, we describe how YARAA configures this mechanism.

During the time YARAA does not detect enough contention, it keeps the same multi-rate retry configuration of SampleRate, which is shown in the first column (no contention) of the Table 3. When our algorithm infers that the contention is the reason for the losses, it changes the multi-rate retry to transmit at the highest successful PHY rate for two consecutive times, falling back to the current PHY rate in the case of failure (Table 3, 2nd column). The transmission rate recommended by the **last statistics** can be different or equal to the **current** PHY rate, depending on

Table 3 YARAA's multi-rate retry chains

	Contention	
	No	Yes
r_0/c_0	Last statistics/2	Highest successful/2
r_1/c_1	Current/3	Current/6
r_2/c_2	Lowest/3	–
r_3/c_3	–	–

Table 4 ORBIT nodes setup

Item	Value
Wireless interface	AR5212 based mini-PCI 802.11a/g
PHY	IEEE 802.11a
Channel	36
Output power	17 dBm
Packet size	1350 bytes
Operating system	GNU/Linux—kernel 2.6.22-3-686
Driver	MadWifi svn.r3366

recent measurements in the **current** and in other rates. These other PHY rates are evaluated by probes which are sent once every ten frames in the **current** rate. As described previously, the recent highest PHY rate is monitored and, if it has poor performance, then another transmission rate is chosen. The performance of a PHY rate is labeled poor when four consecutive frames are lost and this rate is the **current**. When the performance of a transmission rate degrades to this level, it cannot be chosen during a length of time of at least 10 s (quarantine time). Therefore, the **highest successful** is the highest PHY rate not recently labeled poor.

YARAA was developed and initially tested in our laboratory. However, due to the limited number of stations, these tests were aimed mainly at code checking and evaluation of basic functionality.

Afterward, we performed field trials of YARAA in the large network of the Open Access Research Testbed for Next-Generation Wireless Networks (ORBIT) Lab [45]. ORBIT is a testbed that consists of an indoor radio grid for controlled experimentation. The ORBIT testbed has 400 nodes, most of them using Atheros chipsets, which can be fully controlled by remote access. Table 4 summarizes the important items of our ORBIT setup.

For these sets of tests, we established an infrastructure network where the AP was positioned as close to the center of the grid as possible. The stations were located spirally around the AP. Occasionally, some nodes presented problems and were replaced by others, and hence the exact position of the AP and stations could change from one test to another. However, this difference is not relevant because

the distances are small between the nodes, and all of them perceive a high SINR regardless of their position in the grid. In these tests, the traffic is UDP in the upstream direction, and the number of stations varies in order to generate different levels of contention.

The previous tests performed in our laboratory confirmed the initial **diff_time** results obtained from simulation. In the ORBIT testbed, we decided to evaluate the quality of **diff_time** for low traffic stations; i.e., stations that collect far fewer samples than the saturated stations. Figure 8 shows how **diff_time** varies as function of the contention level. In every experiment, the number of stations is indicated on the X-axis, plus one probe station. The probe station sent packets at low rates of 1 packet every second (1 pps), every 5 s (0.2 pps), or every 10 s (0.1 pps). Each test lasted for 300 s and the figure presents the average **diff_time**. As the number of stations increased, the mean **diff_time** also tended to increase. The **diff_time** variance was high, as was observed in the simulation results. As expected, the highest variance occurred when the station transmitted probes the least frequently. In general, regardless the probe rate and despite the variance, **diff_time** is a good indicator for the level of contention.

Finally, we verified the performance of YARAA in comparison with other rate adaptation mechanisms. Besides YARAA, the following algorithms were evaluated: Adaptive Multi-Rate Retry (AMRR) [33], SampleRate [9], and Onoe [36]. AMRR is the implementation of Adaptive ARF (AARF) for the MadWifi driver. AARF presents some improvement over ARF because it increases the threshold used to decide when to switch to an upper rate if the last try was unsuccessful. Onoe (named after its creator, Atsushi Onoe) was the first rate algorithm implemented in the MadWifi driver. This algorithm is based on the loss rate of the last one second, and tries to find the highest PHY rate that has more than 50% success. In our setup, the SINR

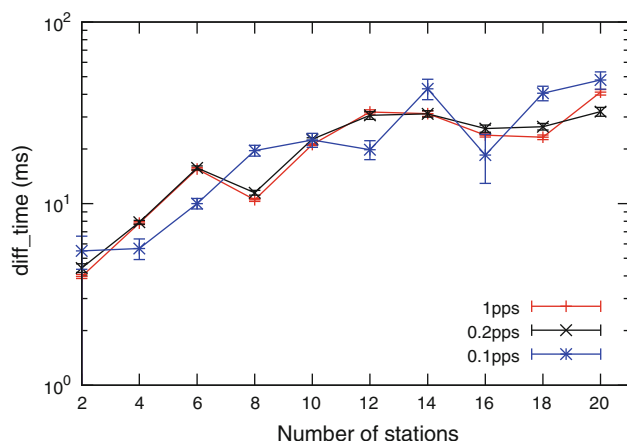


Fig. 8 Values of **diff_time** from different probing rates

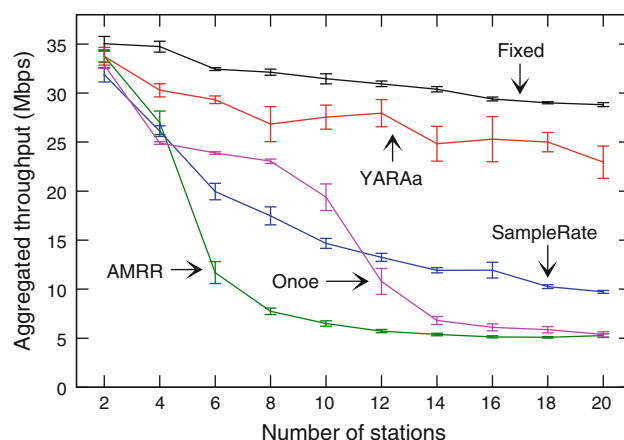


Fig. 9 Aggregated UDP throughput as function of the number of stations

was high enough to transmit at the highest PHY rate (54 Mbps) with very low loss probability due to channel errors. Hence, in this scenario, the best approach is to keep the rate fixed at 54 Mbps, and this strategy (labeled **Fixed**) is included in the evaluation as a replacement for the IDEAL mechanism.

Figure 9 shows the aggregated throughput as a function of the number of stations. AMRR/AARF and SampleRate showed behavior similar to the simulation, but they started degrading earlier and their performances achieved lower values than the simulated ones. Such differences are common, since the simulator represents only a simplified version of the real environment. In this particular situation, the difference came from the absence in the simulator of management frames such as probe request, beacon, association request, etc. While its loss rate was not very high, Onoe sustained an aggregated throughput higher than SampleRate. However, when the frame loss rate increased beyond a certain threshold, Onoe began to make bad PHY rate choices and its performance quickly degraded. As seen in Fig. 9, Onoe was better than SampleRate up to 10 stations, but from 14 onward its performance was almost as poor as AMRR/AARF. Finally, YARAA is the best algorithm at detecting and properly reacting to an increase in contention. With only four stations contending for medium access, YARAA was about 20% better than the other algorithms. As the contention increased, YARAA's advantage increased and became higher than 100% for more than 12 stations.

Some 802.11 manufacturers include extra features in their products to achieve better performance than the competitors. The Atheros devices that were used in our experiments have many enhanced features. Bursting and Fast Frames are features that need attention because they influence YARAA and both are enabled by default. Bursting allows multiple frames to be sent at once, which can

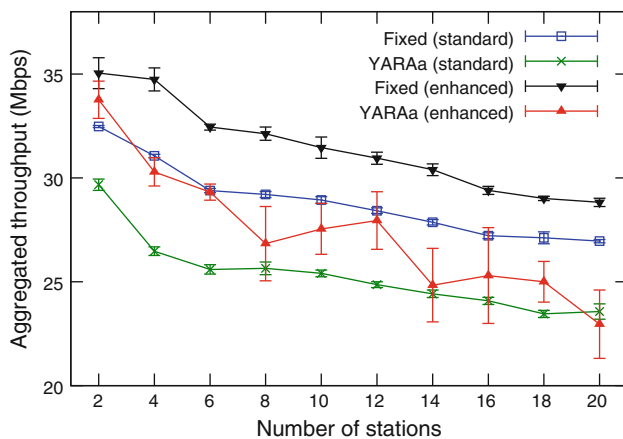


Fig. 10 Enhanced features vs. standard features

reduce the overhead needed for transmission. Fast Frames feature increases the amount of information that can be sent per frame, also resulting in a decrease of transmission overhead. Both features affect the transmission time and so YARAA's measurements. However, the impact on the throughput performance is not high, as shown in Fig. 10. YARAA presents a higher variance under the presence of Bursting and Fast Frames, but throughput is still notably higher than in the other rate adaptation mechanisms. The enhanced features provides some improvement in throughput performance, as can be clearly noticed by comparing Fixed strategy's curves. However, when devices use standard setup, i.e. with both Bursting and Fast Frames disabled, YARAA's behavior is more stable and closely resemble the Fixed one. All results shown below were obtained with these features disabled.

4.3 Additional experiments

This section presents additional performance evaluation of YARAA according to new metrics and in response to

different setups. Initially, the impact on performance of non-optimal settings on YARAA is evaluated. Secondly, the YARAA's accuracy on collision level detection is detailed. At last, the fairness of different rate adaptation mechanisms in comparison to YARAA is evaluated.

4.3.1 YARAA setup

As described in Sect. 3, YARAA compares **diff_time** against two thresholds: **lowThresh** and **highThresh**. Below **lowThresh**, YARAA does not try to increase the rate above the one designated by SampleRate, and above **highThresh**, every frame is transmitted at the present highest successful rate. Between the thresholds, YARAA decides when transmitting at the highest successful rate based on a probability, which increases as **diff_time** becomes closer to **highThresh**. Concerning the default configuration of these thresholds in the driver, we have measured **diff_time** in our testbed under different contention levels and chosen the setting that provides the best performance. In the ns-2 simulator, slightly different values were identified, which have presented limited impact on the performance.

In order to evaluate the influence of thresholds values, several experiments were run in the simulator and some of them were chosen to be rerun in the orbit lab. Figure 11(a) and (b) show the results obtained from ns-2, using the same environment configuration described in Sect. 4.1.2. In this context, **lowThresh** presented more sensitiveness to non-optimal values than **highThresh**. As the value of **lowThresh** increases (Fig. 11a), YARAA defers its reaction to higher levels of contention. Since the occurrences of collision are not detected, the observed losses imply in rate decreasing and so in performance degradation. The setup of different values for **highThresh** (Fig. 11b) has a smaller impact on the performance because YARAA already becomes active since when

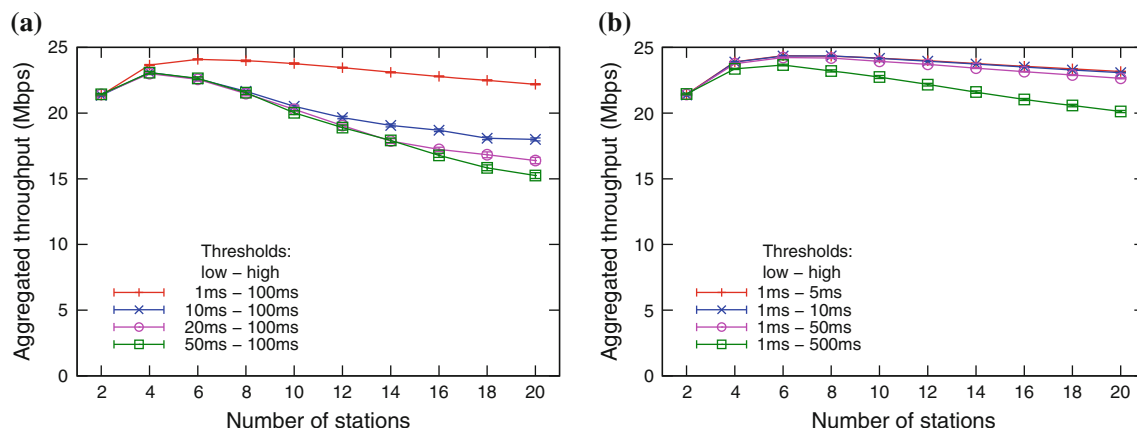


Fig. 11 Influence of different thresholds values on YARAA's performance (simulation). **a** Low threshold, **b** High threshold

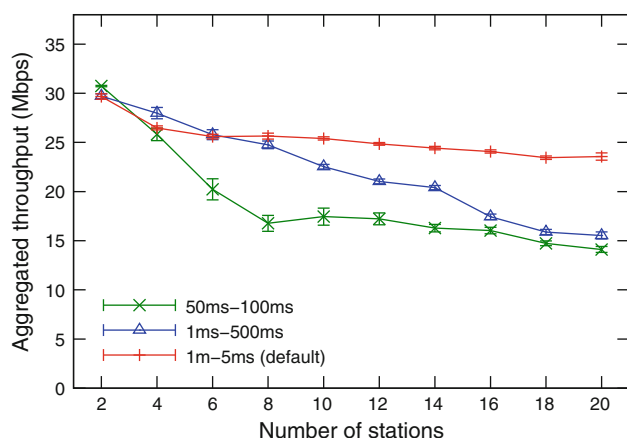


Fig. 12 Influence of different thresholds values on YARAA's performance (field trial)

diff_time is above **lowThresh**, though the probability of trying the current highest successful rate decreases. The thresholds pair 1–10 ms is the default value in the simulator, while 1–5 ms is the default for the driver. As illustrated by Fig. 11(b), the aggregated throughput is the same with both settings.

The worst threshold settings presented in Fig. 11(a) and (b) were chosen to be repeated in the orbit lab. In Fig. 12, the results from those settings and the default ones are shown. It is observed a similar behavior to the one obtained in the simulator.

4.3.2 Contention level detection

Another important performance issue concerns the YARA's ability to detect the current level of contention occurring in the channel. For this purpose, YARAA keeps track of **diff_time** values and computes the probability of sending frames at the highest successful PHY rate. This strategy largely increases the YARAA's robustness against non-optimal configuration settings under good SINR conditions, as shown in the results below. The probability computed by YARAA can be seen as an indicator of the collision rate, which is shown in the simulation results of Figs. 13 and 14.

In Fig. 13, the collision prediction in different frame loss scenarios is presented. The model curve was obtained from [49], in which the authors derive an analytical model for frame collision in a saturated network. Frame loss rate (FLR) is computed in an environment with no loss by signal degradation, so all losses are due to collisions. As expected, FLR is very close to the model, indicating that the simulator properly represents the contention in 802.11 saturated networks. The values of eFCR (estimated frame collision rate) describes how YARAA estimates FLR based on **diff_time**. The figure also shows the estimation

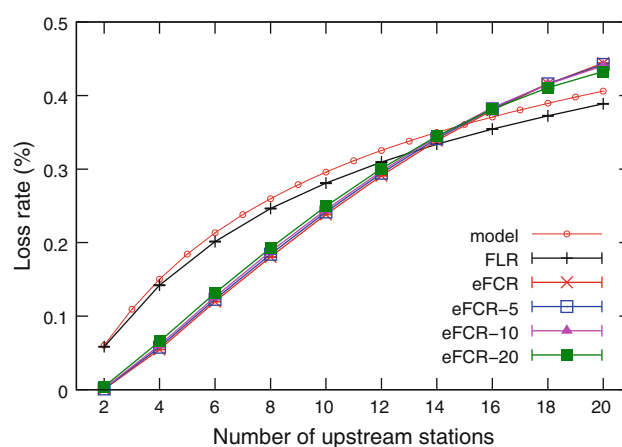


Fig. 13 Performance of the collision prediction (simulation)

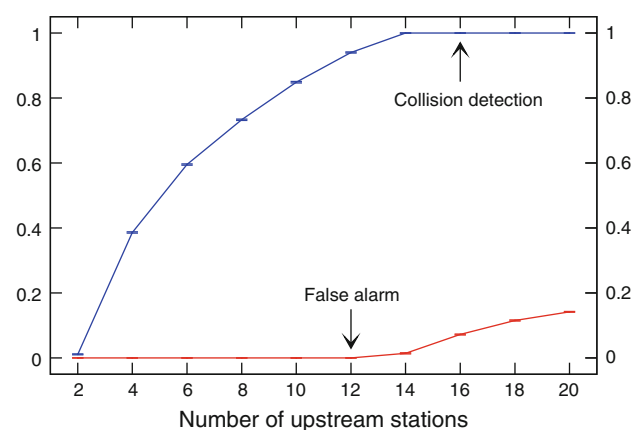


Fig. 14 Collision detection and False alarm (simulation)

behaviour in the presence of additional losses due to signal degradation under three different rates: 5% (eFCR-5), 10% (eFCR-10) 10 and 20% (eFCR-20). To generate these controlled loss rates, we have employed our hidden Markov model (HMM) [10]. This HMM can be adjusted to inject frame losses due to signal degradation, independently of frame collision rate. As YARAA was designed to be implemented in a driver, simplicity is an important goal. This is the reason for eFCR not following FLR very tightly. Also, the impact on performance is low, which does not reward an additional complexity. The values of eFCR-5, eFCR-10 and eFCR-20 illustrate that **diff_time** continues being a good indicator of contention level even under different conditions of signal degradation.

Based on the difference between FLR and eFCR, collision detection and false alarm rates were computed (Fig. 14). Since FLR is obtained on an environment without losses due to signal degradation, it is the actual frame collision rate. Collision detection describes the percentage of collisions that are correctly indicated and false alarm represents the percentage of collisions wrongly detected.

While the contention is low, until 6 stations, collision detection is below 50%. As the contention increases, collision detection becomes more precise, reaching over 80% at 10 stations and 100% at 14 stations. False alarm appears only from 16 stations, increasing slowly and staying below 20% with 20 stations.

As described previously, off-the-shelf 802.11 devices do not provide native resources to identify the reason of a frame loss, i.e. if a collision has occurred or the signal has degraded. In a testbed such as orbit lab, it is reasonable to assume that almost all losses are due to collisions. However, there are two additional issues. First, a collision involves two or more devices, so the sum of all lost frames reported by all nodes is not an accurate measure. Second, physical layer capture (PLC) is not a rare phenomenon and it also affects the collision measurement, because one device can receive a frame even under the presence of strong interference, while other device loses its frame. Despite these inaccuracies, the FLR should follow the trend of the model curve, which is verified in Fig. 15. In the MadWifi driver, a special procedure was required in order to compute the eFCR. As described in Sect. 4.2, multi-rate retry uses an array to configure transmission rates for all tries of a single packet, which means that it is not possible to identify the congestion level in a per frame basis. The solution was to measure eFCR as a percentage of the procedure calls made to setup the PHY rates for a packet. A packet can be retransmitted up to 8 times when YARAA is used as the rate control mechanism (Table 3), so this eFCR measurement is not accurate. However, similar to FLR, eFCR exhibits a well-defined trend, following the model curve. Due to the limitations in measuring accurately these metrics in the driver, collision detection and false alarm rates must not be analyzed by their absolute values, but instead by their trends. Figure 16 confirms that collision detection improves as the contention level increases.

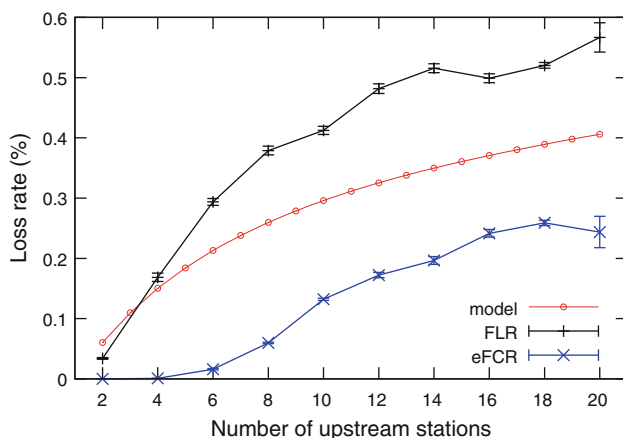


Fig. 15 Performance of the collision prediction (field trial)

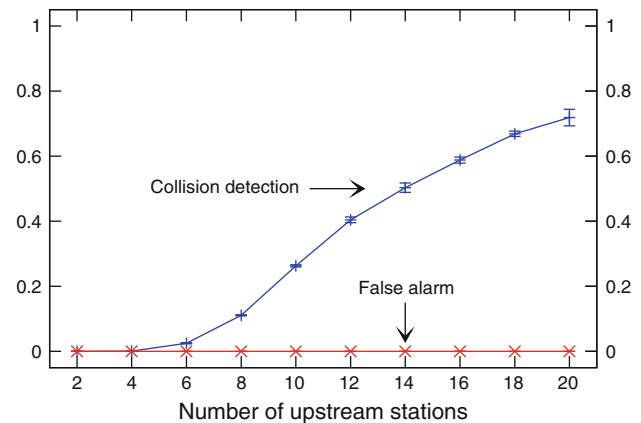


Fig. 16 Collision detection and False alarm (field trial)

Table 5 Fairness comparison for the 20-sender scenario

Rate adaptation mechanism	Avg. JFI	Std. Dev. in JFI
AMRR	0.605	0.007
Onoe	0.872	0.057
SampleRate	0.793	0.023
Fixed (54~Mbps)	0.922	0.004
YARAA	0.977	0.006

4.3.3 Fairness

The last evaluation is about fairness in the sharing of the channel capacity among the contending stations under different rate adaptation mechanisms. The metric used on this evaluation was the Jain's Fairness Index (JFI) [23]. Table 5 reports the mean and std. dev. in JFI for AMRR, Onoe, SampleRate, Fixed and YARAA in an environment with 20 stations. The statistics were obtained from 10 tests of 300 s each. Figure 17 presents the results from one experimental run of each rate adaptation mechanism. This figure shows the mean throughput of individual senders in one experiment. Our results about fairness are similar to the ones presented in [44] except by some details. We have included AMRR into the evaluation and employed the default configuration of Onoe which keeps multi-rate retry (MRR) enabled. In [44], the authors disabled MRR in Onoe tests, which makes the algorithm to use a single rate in all retransmission tries, obtaining some improvement in aggregated throughput but decreasing fairness.

Adaptive multi-rate retry also presents the worst performance in fairness index (0.605). Figure 17(a) presents the widest range of individual senders' throughput, exhibiting differences of until one order of magnitude. Onoe has a conservative strategy to choose PHY rates which implies in low aggregated throughput but a good fairness index (0.872). Similar to [44], Fixed rate shows slight imbalances

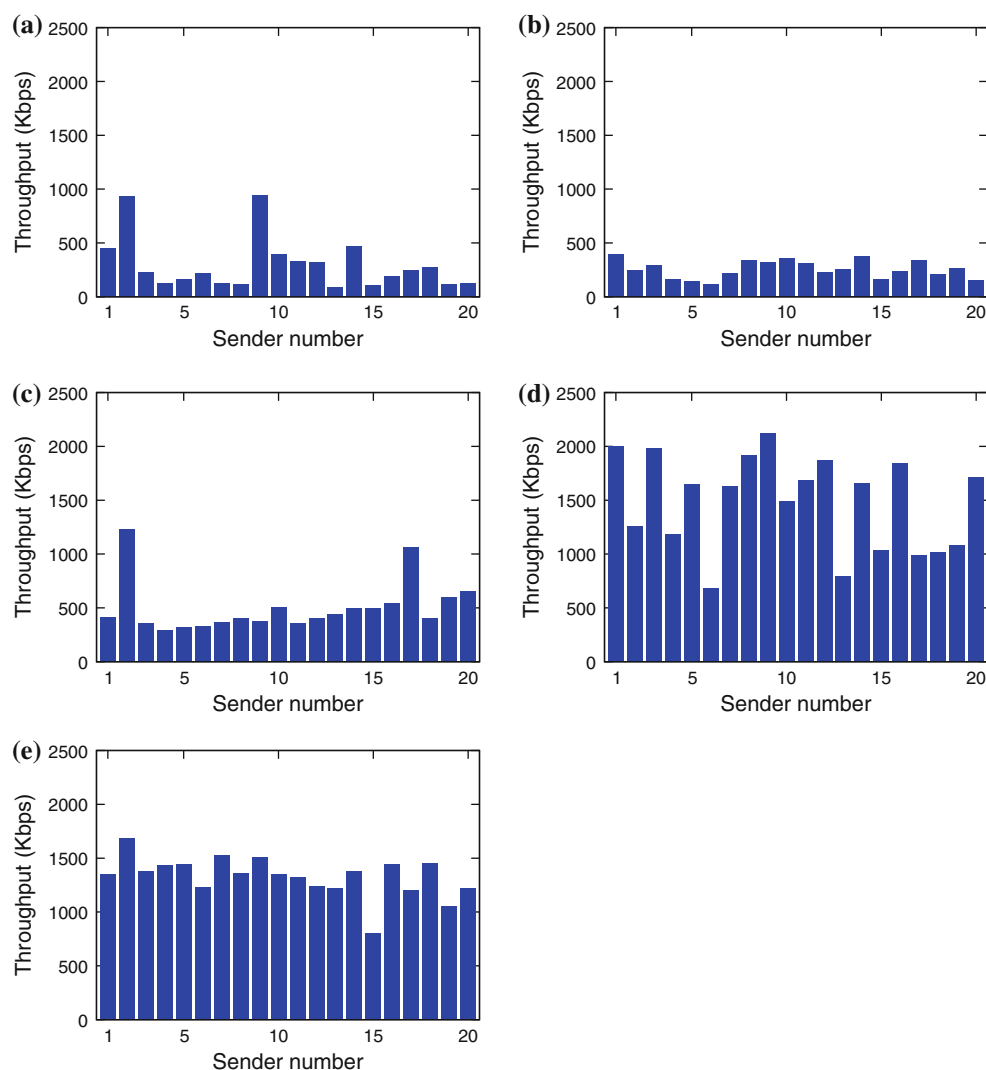


Fig. 17 Throughput fairness of rate adaptation algorithms from one experimental run. **a** AMRR, **b** Onoe, **c** SampleRate, **d** Fixed (54 Mbps), **e** YARAA

(0.922) due to the PLC effect, which is not rare in short range communication environments such as the orbit lab [16, 34]. This effect exposes nearby nodes to different frame loss rates (FLR). SampleRate is also affected by PLC since the mean transmission time is closely related to FLR. As a consequence, there are nodes with noticeable differences in throughput performance, as illustrated in Fig. 17(c), and the fairness index is low (0.793). YARAA presents the best fairness index (0.977) because the differences in FLR are mitigated by choosing different highest transmission rates to the nodes.

5 Conclusion and future work

The algorithms presently used in commercially available 802.11 devices are not able to deal properly with high

contention levels on medium access. Some mechanisms have been proposed, but they violate the IEEE standard or depend on specific hardware support. Many other algorithms were tested only by simulation, and do not take into account real world restrictions. In this context, we presented a new automatic rate adaptation mechanism, which has been extensively evaluated not only in simulation, but also in a large real network. Our mechanism shows good performance even under severe contention.

YARAA is also a promising approach for IEEE 802.11n, but it is necessary real world measurements in order to proper setup the mechanism. Some 802.11n PHY characteristics are similar to .11a (5 GHz) and .11g (2.4 GHz) such as slot time and SIFS. However, 802.11n has more PHY transmission rates and frame aggregation is part of the standard. Besides, YARAA's core is a distributed contention detection technique which has been used in this

work to properly adapt the PHY rate. Nevertheless, this technique can be used by other mechanisms. In future work, we are interested in evaluating the performance of our technique on 802.11n devices, on adaptively enabling and disabling RTS/CTS exchange, and on changing the MAC congestion window size. Another promising application of the technique is in a cross-layer indication of contention which, for example, would be useful for a transport layer.

Acknowledgments This work was supported by CAPES, CNPq, FAPERJ, FINEP and RNP. We thank WINLAB/Rutgers University for granting access to the ORBIT testbed.

References

- Acharya, P. A., Sharma, A., Belding, E. M., Almeroth, K. C., & Papagiannaki, K. (2010). Rate adaptation in congested wireless networks through real-time measurements. *IEEE Transactions on Mobile Computing*, 99 (PrePrints).
- Acharya, P. A. K., Sharma, A., Belding, E. M., Almeroth, K. C., & Papagiannaki, K. (2008). Congestion-aware rate adaptation in wireless networks: A measurement-driven approach. In *IEEE communications society conference on sensor, mesh and ad hoc communications and networks (SECON)*, pp. 1–9.
- Aguayo, D., Bicket, J., Biswas, S., Judd, G., & Morris, R. (2004). Link-level measurements from an 802.11b mesh network. In *ACM special interest group on data communication (SIGCOMM)*, vol. 34, pp. 121–132.
- ARC: Additions to the NS network simulator to handle Ricean and Rayleigh fading. <http://www.ece.cmu.edu/wireless/downloads.html>. Last access Dec 18, 2008.
- Baldo, N., Maguolo, F., Merlin, S., Zanella, A., Zorzi, M., Melpignano, D., & Siorpaes, D. (2008). GORA: Goodput optimal rate adaptation for 802.11 using medium status estimation. In *IEEE international conference on communications (ICC)*, pp. 4916–4921.
- Bianchi, G. (2000). Performance analysis of the IEEE 802.11 distributed coordination function. *IEEE Journal on Selected Areas in Communications*, 18(3), 535–547.
- Biaz, S., & Wu, S. (2008). ERA: Effective rate adaptation for WLANs. In *International IFIP-TC6 networking conference on ad hoc and sensor networks, wireless networks, next generation internet (NETWORKING)*, pp. 901–913.
- Biaz, S., & Wu, S. (2008). Loss differentiated rate adaptation in wireless networks. In *IEEE wireless communications and networking conference (WCNC)*, pp. 1639–1644.
- Bicket, J. C. (2005). *Bit-rate selection in wireless networks*. Master's thesis, Department of Electrical Engineering and Computer Science-Massachusetts Institute of Technology.
- Cardoso, K. V., & de Rezende, J. F. (2009). Accurate hidden Markov modeling of packet losses in indoor 802.11 networks. *IEEE Communications Letters*, 13(6), 417–419.
- Chaporkar, P., Proutiere, A., & Radunovic, B. (2010). Rate adaptation games in wireless LANs: Nash equilibrium and price of anarchy. In *IEEE international conference on computer communications (INFOCOM)*, pp. 2052–2060.
- Cheng Chen, C., Luo, H., Seo, E., Vaidya, N. H., & Wang, X. (2007). Rate-adaptive framing for interfered wireless networks. In *IEEE international conference on computer communications (INFOCOM)*, pp. 1325–1333.
- Choi, J., Na, J., Park, K., & kwon Kim, C. (2007). Adaptive optimization of rate adaptation algorithms in multi-rate WLANs. In *IEEE International conference on network protocols (ICNP)*, pp. 144–153.
- Ergin, M. A., Ramachandran, K., & Gruteser, M. (2008). An experimental study of inter-cell interference effects on system performance in unplanned wireless LAN deployments. *Computer Networks*, 52(14), 2728–2744.
- Floyd, S., & Jacobson, V. (1993). Random early detection gateways for congestion avoidance. *IEEE/ACM Transactions on Networking*, 1(4), 397–413.
- Ganu, S., Ramachandran, K., Gruteser, M., Seskar, I., & Deng, J. (2006). Methods for restoring MAC layer fairness in IEEE 802.11 networks with physical layer capture. In *International workshop on multi-hop ad hoc networks: From theory to reality (REALMAN)*, pp. 7–14.
- Ha, J., Lee, K., Kim, H., & Kang, I. (2008). A snooping rate adaptation algorithm for IEEE 802.11 WLANs. In *International Symposium on wireless pervasive computing (ISWPC)*, pp. 606–609.
- Haratcherev, I., Langendoen, K., Legendijk, R., & Sips, H. (2004). Hybrid rate control for IEEE 802.11. In *ACM International workshop on mobility management & wireless access protocols (MobiWac)*, pp. 10–18.
- Ho, C. K., Oostveen, J., & Linnartz, J. P. M. G. (2009). Rate adaptation using acknowledgement feedback in finite-state Markov channels with collisions. *IEEE Transactions on Wireless Communications*, 8(6), 3226–3239.
- Hoffmann, C., Manshaei, M. H., & Turetli, T. (2005). CLARA: Closed-loop adaptive rate allocation for IEEE 802.11 wireless LANs. In *IEEE international conference on wireless networks, communications and mobile computing (WirelessCom)*, vol. 1, pp. 668–673.
- Holland, G., Vaidya, N., & Bahl, P. (2001). A Rate-adaptive MAC protocol for multi-hop wireless networks. In *ACM international conference on mobile computing and networking (MobiCom)*, pp. 236–251.
- IEEE 802.11. (1999). Edition (ISO/IEC 8802-11: 1999). *IEEE standards for information technology—telecommunications and information exchange between systems—local and metropolitan area network—specific requirements—Part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) Specifications*, 1999.
- Jain, R. K., Chiu, D. M. W., & Hawe, W. R. (1984). *A quantitative measure of fairness and discrimination for resource allocation in shared computer system*. Tech. rep., Digital Equipment Corporation.
- Ji, Z., Yang, Y., Zhou, J., Takai, M., & Bagrodia, R. (2004). Exploiting medium access diversity in rate adaptive wireless LANs. In *ACM international conference on mobile computing and networking (MobiCom)*, pp. 345–359.
- Joshi, T., Ahuja, D., Singh, D., & Agrawal, D. P. (2008). SARA: Stochastic automata rate adaptation for IEEE 802.11 networks. *IEEE Transactions on Parallel and Distributed Systems*, 19(11), 1579–1590.
- Judd, G., Wang, X., & Steenkiste, P. (2008). Efficient channel-aware rate adaptation in dynamic environments. In *ACM international conference on mobile systems, applications, and services (MobiSys)*, pp. 118–131.
- Kamerman, A., & Monteban, L. (1997). WaveLAN-II: A high-performance wireless LAN for the unlicensed band. *Bell System Technical Journal*, 2(3), 118–133.
- Kim, J., Kim, S., Choi, S., & Qiao, D. (2006). CARA: Collision-aware rate adaptation for IEEE 802.11 WLANs. In *IEEE international conference on computer communications (INFOCOM)*, pp. 1–11.

29. Kim, S., Lee, S. J., & Choi, S. (2006). The impact of IEEE 802.11 MAC strategies on multi-hop wireless mesh networks. In *IEEE workshop on wireless mesh networks (WiMesh)*, pp. 38–47.
30. Koçi, N., & Marina, M. K. (2009). Understanding the role of multi-rate retry mechanism for effective rate control in 802.11 wireless LANs. In *IEEE local computer networks*, 2009, pp. 305–308.
31. Kumar, A., Altman, E., Miorandi, D., & Goyal, M. (2005). New insights from a fixed point analysis of single cell IEEE 802.11 WLANs. In *IEEE International conference on computer communications (INFOCOM)*, pp. 1550–1561.
32. Kurth, M., Zubow, A., & Redlich, J. P. (2006). Multi-channel link-level measurements in 802.11 mesh networks. In *International conference on wireless communications and mobile computing (IWCMC)*, pp. 937–944.
33. Lacage, M., Manshaei, M. H., & Turetli, T. (2004). IEEE 802.11 rate adaptation: A practical approach. In *ACM international Symposium on modeling, analysis and simulation of wireless and mobile systems (MSWiM)*, pp. 126–134.
34. Lee, J., Kim, W., Lee, S. J., Jo, D., Ryu, J., Kwon, T., & Choi, Y. (2007). An experimental study on the capture effect in 802.11a networks. In *International workshop on wireless network testbeds, experimental evaluation and characterization (WinTECH)*, pp. 19–26.
35. Lv, S., Wang, X., Zhou, X., & Liu, C. (2007). Efficient rate adaptation in multi-rate 802.11 networks. In *IEEE Symposium on computers and communications (ISCC)*, pp. 545–550.
36. MadWifi: MadWifi. <http://madwifi-project.org>. Last access Dec 18, 2008.
37. Mhatre, V. (2007). Enhanced wireless mesh networking for ns-2 simulator. *ACM SIGCOMM Computer Communication Review*, 37(3), 69–72.
38. Ngugi, A. N., Chen, Y. P., & Li, Q. (2009). Rate adaptation with NAK-aided loss differentiation in 802.11 wireless networks. In *IEEE global communications conference (GLOBECOM)*, pp. 3073–3078.
39. Pang, Q., Leung, V. C., & Liew, S. C. (2005). A rate adaptation algorithm for IEEE 802.11 WLANs based on MAC-layer loss differentiation. In *International conference on broadband networks (BROADNETS)*.
40. Papagiannaki, K., Yarvis, M., & Conner, W. S. (2006). Experimental characterization of home wireless networks and design implications. In *IEEE international conference on computer communications (INFOCOM)*, pp. 1–13.
41. del Prado Pavon, J., & Choi, S. (2003). Link adaptation strategy for IEEE 802.11 WLAN via received signal strength measurement. In *IEEE international conference on communications (ICC)*, vol. 2, pp. 1108–1113.
42. Qiao, D., & Choi, S. (2005). Fast-responsive link adaptation for IEEE 802.11 WLANs. In *IEEE international conference on communications (ICC)*, vol. 5, pp. 3583–3588.
43. Qiao, D., Choi, S., & Shin, K. G. (2002). Goodput analysis and link adaptation for IEEE 802.11a wireless LANs. *IEEE Transactions on Mobile Computing*, 1(4), 278–292.
44. Ramachandran, K., Kremo, H., Gruteser, M., Spasojević, P., & Šeškar, I. (2007). Experimental scalability analysis of rate adaptation techniques in congested IEEE 802.11 networks. In *IEEE international Symposium on world of wireless, mobile and multimedia networks (WoWMoM)*.
45. Raychaudhuri, D., Seskar, I., Ott, M., Ganu, S., Ramachandran, K., Kremo, H., Siracusa, R., Liu, H., & Singh, M. (2005). Overview of the ORBIT radio grid testbed for evaluation of next-generation wireless network protocols. In *Wireless communications and networking conference (WCNC)*, vol. 3, pp. 1664–1669.
46. Sadeghi, B., Kanodia, V., Sabharwal, A., Knightly, E. (2005). OAR: An opportunistic auto-rate media access protocol for ad hoc networks. *Kluwer Academic Wireless Networks*, 11(1–2), 39–53.
47. SIGNET: dei80211mr: A new 802.11 implementation for NS-2. <http://www.dei.unipd.it/wdyn/?IDsezione=50.0>. Last access Dec 18, 2008.
48. Song, Y., Zhu, X., Fang, Y., & Zhang, H. (2010). Threshold optimization for rate adaptation algorithms in IEEE 802.11 WLANs. *IEEE Transactions on Wireless Communications*, 9(1), 318–327.
49. Tay, Y. C., & Chua, K. C. (2001). A capacity analysis for the IEEE 802.11 MAC protocol. *Wireless Networks* 7(2), 159–171.
50. Vutukuru, M., Balakrishnan, H., & Jamieson, K. (2009). Cross-layer wireless bit rate adaptation. *ACM SIGCOMM Computer Communication Review*, 39(4), 3–14.
51. Wang, S. C., & Helmy, A. (2008). BEWARE: background traffic-aware rate adaptation for IEEE 802.11. In *International Symposium on a world of wireless, mobile and multimedia networks (WoWMoM)*, pp. 1–12.
52. Wong, S. H. Y., Yang, H., Lu, S., & Bharghavan, V. (2006). Robust rate adaptation for 802.11 wireless networks. In *ACM International conference on mobile computing and networking (MobiCom)*, pp. 146–157.
53. Xia, Q., & Hamdi, M. (2008). Smart sender: A practical rate adaptation algorithm for multirate IEEE 802.11 WLANs. *IEEE Transactions on Wireless Communications*, 7(5), 1764–1775.
54. Zhang, J., Tan, K., Zhao, J., Wu, H., & Zhang, Y. (2008). A practical SNR-guided rate adaptation. In *IEEE International conference on computer communications (INFOCOM)*, pp. 2083–2091.

Author Biographies



Kleber V. Cardoso received the B.Sc. degree in Computer Science from Universidade Federal de Goiás (UFG) in 1997. He received the M.Sc. and Ph.D. degrees in Electrical Engineering from the Universidade Federal do Rio de Janeiro (UFRJ) in 2002 and 2009, respectively. Since 2009, he is a Professor at Universidade Federal de Goiás. His research interests include wireless networks, internet protocols, quality of service, and mobile networks.



José F. de Rezende received the B.Sc. and M.Sc. degrees in Electronic Engineering from the Universidade Federal do Rio de Janeiro (UFRJ) in 1988 and 1991, respectively. He received the Ph.D. degree in Computer Science from the Université Pierre et Marie-Curie, France, in 1997. He was an associate researcher at LIP6 (Laboratoire d'Informatique de Paris 6) during 1997. Since 1998, he is an Associate Professor at Universidade Federal do Rio de Janeiro.

His research interests include distributed multimedia applications, multipeer communication, wireless communication, quality of service issues on high speed and mobile networks. He is an associate editor of the Elsevier Ad Hoc Networks Journal.